

AI Architecture from the Ground Up

*A practical guide to designing agent systems — for anyone
building with AI: the layers, the patterns, the honest
evidence, and a method for your own stack*

A practical field guide

2026

By **Christian Macion** — AI engineer & quantitative researcher. Built on adversarially fact-checked research: two deep-research passes on the agent-AI literature (53 sources, several claims refuted and dropped) plus a docs-level reference of the major agent frameworks. Tool-agnostic; examples drawn from Claude Code, the OpenAI Agents SDK, and LangGraph. Share freely.

START HERE — THIS BOOK IS FOR BEGINNERS

You do not need to know anything about AI to read this. If you have used a chatbot, you have enough. Every term is defined the first time it appears, always with a plain example before the abstract name. If a page ever feels heavy, you are allowed to slow down — the book is built in small pieces on purpose.

The format itself is based on how people actually learn (cognitive-load theory; Mayer's multimedia principles; the 'testing effect'). Four promises that follow from that:

- 1. Scannable.** Every chapter opens with a one-line navy **BOTTOM LINE**. Read only those and you still get the whole spine.
- 2. Small chunks.** No section throws more than a few new ideas at you at once — that is the limit of working memory, not a lack of content.
- 3. Shown, then faded, then checked.** A worked example first, then the same idea with blanks, then a quick recall.
- 4. You'll be quizzed — gently.** Small **CHECK YOURSELF** boxes ask you to remember before moving on, because recalling beats re-reading. The answers sit right below; cover them first.

A NOTE ON EVIDENCE (READ THIS)

The single most valuable habit in AI engineering is the same as in quant research: **do not dress a guess as a measurement**. Every factual claim and figure carries a tag.

V Verified — checked against a primary source (Anthropic / OpenAI / LangChain / a peer-reviewed paper), and in this book additionally *adversarially* verified: each headline number survived a 3-vote refutation pass in the deep-research harness.

B Background — established, docs-grounded fact (e.g. a Claude Code primitive) stated without a fresh citation.

SCHEMATIC Schematic — an illustrative diagram or synthetic worked case. The shape is real; the exact numbers are for teaching, not measured.

Where the evidence is genuinely two-sided, this book says so. That honesty is the point.

THE PARTS OF EVERY SECTION

Sections run a simple engine (not every part every time): **Orient** (the big picture, often the question we're answering) → **Prime** (the few new terms) → **Build** (the explanation) → **Show** (a worked case) → **Fade** (you fill the blanks) → **Check Yourself** (recall from memory) → **Recap** (back to altitude) → **Glossary**. Boxes are colour-coded so you can find the part you want at a glance — and so you can skip what you already know.

MAP OF THE WHOLE BOOK

Part A — Foundations: the layered stack. What an agent actually is, the three layers, and why you escalate slowly.

Part B — Multi-agent architecture. The three topologies, the four patterns, the two-sided evidence, swarm reality, and the framework landscape.

Part C — Building it. The primitives (sub-agents, skills, workflows, memory), model routing, cost, and context engineering — tool-agnostic, with real examples.

Part D — Designing your own stack. A repeatable five-step method to audit and architect any agent system, sequence the build, and add the governance that prevents failure.

Part E — How the pros work. What practitioners actually do — the verification habit, a hard-won case study from finance, and the meta-practices that scale.

Close — Where this leaves us. The one-sentence verdict and the evidence base.

Part A. Foundations: The Layered Stack

Before you can design a system of agents you need a precise picture of what *one* agent is, and why stacking more of them is a decision with real costs. This Part builds the vocabulary — foundation model, agent, tool, the three layers — and lands the single most important discipline in the field: **escalate slowly**. Add capability only when the cheaper rung has demonstrably run out.

Chapter 1. Agents, tools, and the three layers

BOTTOM LINE An **agent** is a foundation model placed inside a loop with tools and memory; the modern stack is just three layers — model, single agent, many agents — each wrapping the one below and each costing more than it.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Define foundation model, agent, and tool without hand-waving.
- Name the three layers of the agentic stack and what each adds.
- Explain why ‘more agents’ is a cost decision, not a default.
- Map the layers onto a workflow you already run by hand.

Prerequisites: comfort with the idea of an LLM; no prior agent-framework experience needed.

1.1 What sits at the bottom: the foundation model

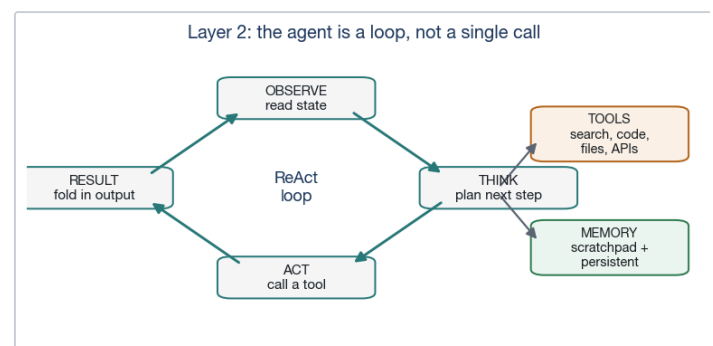
ORIENT: *the big picture before the detail*

A foundation model (Opus, Sonnet, a GPT, a Llama) does exactly one thing: given text, it predicts the next token. It has no memory of yesterday, cannot run code, cannot read a file. It is brilliant and inert — a brain in a jar. Everything ‘agentic’ is scaffolding we build *around* this core.

PRIME: *new terms, one line each*

- **Foundation model:** the raw next-token predictor; Layer 1. No tools, no memory, no loop.
- **Tool:** a function the model can call — web search, code execution, a file read, an API.
- **Agent:** a model placed in a loop where it can call tools and observe results until a goal is met.
- **Context window:** the finite text the model can ‘see’ at once; its entire working memory.

The jump from a model to an **agent** is the jump from a single question-and-answer to a *loop*. An agent observes the current state, decides on a step, takes an action (usually a tool call), folds the result back into its context, and repeats until the task is done or it gives up. That loop — often called **ReAct** (reason + act) — is the whole trick. Reflection and planning are refinements of it: reflection adds a ‘critique your own last step’ turn; planning adds a ‘write the steps first’ turn.



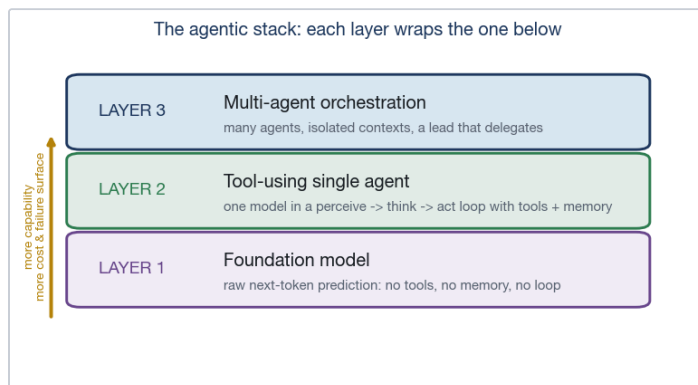
SCHEMATIC Layer 2: an agent is a loop around the model, not a single call. Tools and memory hang off the loop.

1.2 The three layers

ORIENT: *the big picture before the detail*

Almost every ‘AI architecture diagram’ you will ever see is a rearrangement of three layers. Hold them in your head and the field stops being a fog of buzzwords.

Layer 1 — foundation model. The brain in the jar. **Layer 2 — the tool-using single agent.** One model in a loop with tools and memory; this is where most real value is captured, and where you should start. **Layer 3 — multi-agent orchestration.** Several agents, usually with their own isolated contexts, coordinated by a lead that delegates. Each layer adds capability — and adds cost and new ways to fail. The arrow up the side of the diagram below is the through-line of this entire book: capability and fragility rise together.



SCHEMATIC The agentic stack. Layer 3 wraps Layer 2 wraps Layer 1; cost and failure surface grow as you climb.

COMMON MISTAKE: *you might think X; actually Y*

You might think Layer 3 is simply ‘better’ — the advanced version you should aim for. Actually each layer is the *right* answer for a different job. A huge share of production AI value lives entirely at Layer 2. Climbing to Layer 3 when Layer 2 would do is the single most common — and expensive — architecture mistake in the field.

1.3 You already think in these layers

BRIDGE: *from what you already own*

You may already run a Layer-3 system without calling it that. Any time you split a job across ‘a researcher, a drafter, and an editor’ — each focused on one thing, each handing a clean result to the next — you are using the orchestrator-worker shape this book will name and ground in the literature. The goal here is to make the instinct explicit, so you can extend it deliberately instead of by feel.

CHECK YOURSELF: *cover the answers, recall from memory*

From memory, before you read on:

1. What does an agent add that a bare foundation model lacks?
2. Name the three layers of the stack, bottom to top.
3. Why is jumping to Layer 3 (many agents) a risky default?

Answers: (1) a loop, plus tools and memory — it can act and remember, not just answer once. (2) Layer 1 foundation model, Layer 2 single tool-using agent, Layer 3 multi-agent orchestration. (3) every layer up adds cost and new ways to fail; most value lives at Layer 2, so climbing without need is the common expensive mistake.

RECAP: *back to altitude*

- A foundation model predicts tokens; an agent is that model in a tool-using loop.
- Three layers: model (L1), single agent (L2), multi-agent orchestration (L3).
- Each layer up adds capability AND cost AND failure surface — never climb for its own sake.
- You may already run a Layer-3 system by hand; this book makes the instinct deliberate.

SECTION GLOSSARY

ReAct: an agent loop that interleaves reasoning steps with tool actions. **Orchestrator-worker:** a lead agent that plans and delegates to specialist sub-agents.

Context window: the model’s finite working memory, measured in tokens.

Chapter 2. The single-agent core and pragmatic escalation

BOTTOM LINE The consensus discipline from both Anthropic and LangChain is the same: **start single-agent, add tools before you add agents, and go multi-agent only when you hit a clear limit** — because the cost climbs faster than the capability.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- State the pragmatic-escalation rule and the order of its rungs.
- Recognise the limits that actually justify going multi-agent.
- Avoid the ‘tool sprawl’ and ‘premature orchestra’ traps.

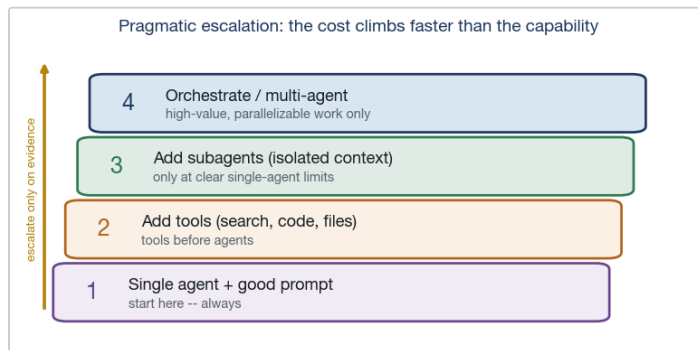
Prerequisites: Chapter 1.

2.1 Tools before agents

ORIENT: *the big picture before the detail*

When a single agent struggles, there are two very different fixes, and engineers reach for the wrong one constantly. Fix A: give the agent a better tool or a sharper prompt. Fix B: split the work across multiple agents. Fix A is cheaper, more reliable, and the right first move almost every time.

The reason is leverage. A well-chosen tool — a code interpreter, a retrieval function, a validated calculator — removes an entire *class* of failure (the model no longer has to do arithmetic in its head). A second agent removes nothing on its own; it adds coordination overhead and a fresh way to miscommunicate. So the escalation order is strict: **better prompt** → **better tools** → **sub-agents** → **full orchestration**. You move up a rung only when the rung you are on is *demonstrably* exhausted — not when you suspect it might be.



SCHEMATIC Pragmatic escalation. Each rung is justified only by evidence the rung below has run out.

2.2 What counts as a real limit

So when is a single agent genuinely out of room? Three honest signals. **Context pressure** **B**: the task needs more material in the window than the model can attend to reliably (Chapter 7’s ‘context rot’). **Genuine parallelism**: the work splits into independent strands that could run at the same time — the case where multi-agent demonstrably wins

(Chapter 4). **Distinct expertise / permissions**: sub-tasks need different tools, system prompts, or trust levels (a read-only adversary vs. a writer). If none of these is true, a second agent is decoration.

SHOW: *worked example (skip if confident)*

You ask one agent to ‘survey twelve volatility-forecasting papers and summarise the consensus.’ It reads three, fills its context with quotations, and the summary of the last nine degrades. This is *context pressure* plus *genuine parallelism* — the two limits stacked. The fix is Layer 3: one sub-agent per paper, each returning a 200-word structured note, a lead that synthesises the twelve notes. The lead never holds twelve full papers at once. (This is precisely Anthropic’s research-system result in Chapter 4.)

FADE: *same skill, you fill the blanks*

Decide single- or multi-agent for each. (a) Reformat 4,000 trade records to a new schema. → _____. (b) For each of 20 instruments, independently scan for a calendar anomaly, then rank them. → _____. (c) Draft one memo from one backtest. → _____.

Answers: (a) single agent + a tool — mechanical, no parallel reasoning; (b) multi-agent — 20 independent strands, real parallelism; (c) single agent — one coherent narrative, splitting it only adds seams.

COMMON MISTAKE: *you might think X; actually Y*

You might think adding tools is always safe. Actually *tool sprawl* is its own failure: twenty half-relevant tools bloat the context and make the model hesitate over which to call. Curate tools as ruthlessly as you would agents — few, sharp, well-described.

RECAP: *back to altitude*

- Escalate in order: prompt → tools → sub-agents → orchestration.
- Three limits justify multi-agent: context pressure, genuine parallelism, distinct expertise/permissions.
- If none holds, a second agent only adds coordination cost and new failure modes.
- Tools beat agents as a first fix — but curate tools too; sprawl is real.

SECTION GLOSSARY

Pragmatic escalation: the discipline of adding architectural complexity only at a proven limit. **Tool sprawl:** degradation from offering an agent too many marginally-useful tools.

Part B. Multi-Agent Architecture

Once you have earned your way to Layer 3, the design space is smaller than it looks. This Part shows that multi-agent systems reduce to **three topologies** and a handful of **patterns**, each with a concrete trade-off. Then it confronts the evidence honestly: multi-agent can beat single-agent by 90% on the right task and fail 40–87% of the time on the wrong one. Finally it deflates the word ‘swarm’ and tours the frameworks.

Chapter 3. Topologies and patterns

BOTTOM LINE Every multi-agent system is a **chain**, a **star**, or a **mesh**; LangChain’s four reusable patterns — subagents, skills, handoffs, routers — are how you actually build them, and each carries one concrete cost.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Draw the three canonical topologies and name a system of each.
- Match the four LangChain patterns to the topologies.
- State the specific trade-off each pattern imposes.

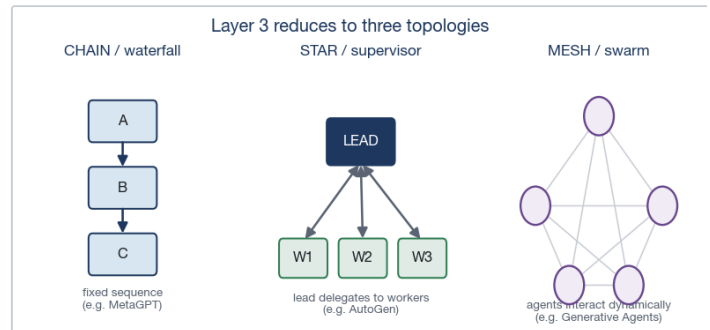
Prerequisites: Part A.

3.1 The three topologies

ORIENT: *the big picture before the detail*

A 2026 architecture survey collapses the zoo of multi-agent designs into three shapes. Learn these and every framework diagram becomes legible.

Chain / waterfall. Agents in a fixed sequence, each handing to the next (MetaGPT’s assembly-line of roles). Predictable, easy to debug, but as rigid as its weakest link. **Star / supervisor.** A central lead delegates to workers and collects results (AutoGen, and Anthropic’s research system). The dominant production shape — it isolates context and parallelises. **Mesh / swarm.** Agents talk to each other dynamically with no fixed hierarchy (the ‘Generative Agents’ simulation). Maximally flexible, hardest to control, most prone to the failure modes in Chapter 4.



V Layer 3 reduces to three topologies. Star/supervisor is the workhorse of production systems.

COMMON MISTAKE: *you might think X; actually Y*

You might think tree/hierarchical is a fourth fundamental shape. It is real and common, but it is a star-of-stars — supervisors managing sub-supervisors. The three shapes are the primitives; everything else composes them.

3.2 The four patterns you actually build with

ORIENT: *the big picture before the detail*

Topologies are the *shape*; patterns are the *mechanism*. LangChain’s four-pattern taxonomy is the cleanest working vocabulary, and each pattern’s weakness is as important as its strength.

The four patterns and their costs:

Pattern	What it is	The concrete cost
Subagents	a supervisor calls specialists as tools	one extra model call per hop (results route back through the lead)
Skills	load specialised prompts/knowledge on demand	context accumulates → token bloat on later calls
Handoffs	the active agent changes with the conversation	stateful — needs careful state management
Routers	classify the input, dispatch to one specialist	stateless & consistent, but pays routing overhead every call

Notice the pattern beneath the patterns: **there is no free coordination**. Subagents buy isolation with an extra call; skills buy flexibility with token growth; handoffs buy adaptivity with state; routers buy consistency with overhead.

Choosing an architecture is choosing *which* cost you are willing to pay, not whether you pay one.

RECAP: back to altitude

- Three topologies: chain (sequence), star (supervisor+workers), mesh (dynamic).
- Star/supervisor is the production workhorse; tree is a star-of-stars.
- Four patterns: subagents, skills, handoffs, routers — each with a fixed cost.
- There is no free coordination; you choose which cost to pay.

SECTION GLOSSARY

Supervisor: the lead agent in a star topology that delegates and synthesises. **Router:** a classify-then-dispatch step sending each input to one specialist. **Handoff:** transfer of the active role from one agent to another mid-task.

Chapter 4. The honest evidence: when multi-agent wins and when it fails

BOTTOM LINE Multi-agent beat single-agent by **90.2%** on breadth-first research — but at **~15× the tokens**, with ~80% of the gain explained by that spend; meanwhile production multi-agent frameworks fail **41–86.7%** of the time. It is task-conditional, not a free win.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Quote both sides of the evidence accurately, with their caveats.
- Identify the task profile where multi-agent genuinely wins.
- Name the three buckets of multi-agent failure and defend against each.
- Resist the claims that did NOT survive verification.

Prerequisites: Part A; Chapter 3.

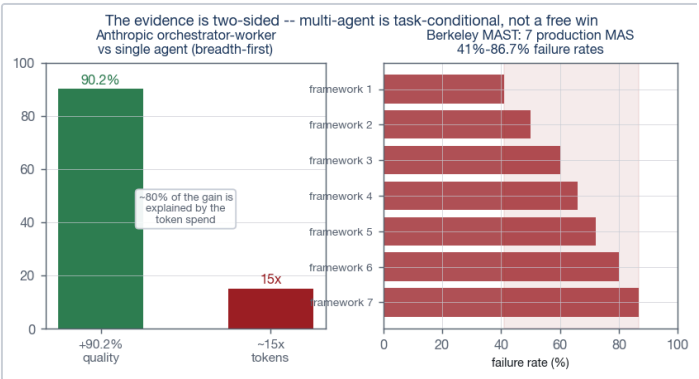
4.1 The win, stated carefully

ORIENT: the big picture before the detail

There is one widely-cited result in favour of multi-agent, and it is strong — but only if you keep its conditions attached. Strip the conditions and you will over-build.

Anthropic’s research system — an Opus lead delegating to Sonnet subagents in a star topology — outperformed a single Opus agent by **90.2%** on their internal research evaluation **v**. The mechanism is exactly Chapter 1’s loop made parallel:

each subagent explores one direction in its own clean context and returns a 1–2k-token summary, so the lead reasons over many directions at once without drowning in detail. **But three caveats travel with that number, and you must carry all three:** it is a *vendor internal* eval, not peer-reviewed; it is scoped to *breadth-first* research (many independent strands); and the same work cost roughly **15× the tokens** of a single chat, with token spend alone explaining about 80% of the measured gain **v**. The lesson is not ‘multi-agent is better.’ It is ‘multi-agent buys parallel breadth, and you pay for it in tokens.’



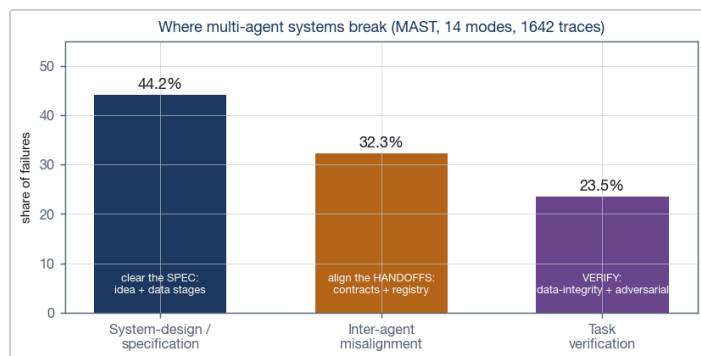
v Both sides at once. Left: the breadth-first win and its token cost. Right: the MAST failure band across 7 production frameworks.

4.2 The failure case, stated just as carefully

ORIENT: *the big picture before the detail*

The counterweight is peer-reviewed and sobering. Most multi-agent systems in the wild do not fail gracefully; they fail often, and for nameable reasons.

The Berkeley **MAST** study ('Why Do Multi-Agent LLM Systems Fail?', NeurIPS 2025) measured **41% to 86.7%** failure rates across seven state-of-the-art open-source multi-agent frameworks, and found their gains over a single agent — or even over simple best-of-N sampling — were often minimal **V**. It catalogued **14 failure modes** in three buckets **V**: **system-design / specification** issues (44.2% — vague roles, unclear task specs), **inter-agent misalignment** (32.3% — agents talking past each other, dropped hand-offs), and **task-verification** failures (23.5% — nobody checked the output). The distribution is the design brief: most pain is in *specification* and *hand-offs*, not in the models themselves.



V Where multi-agent breaks. The three buckets map directly onto defences you can build.

BRIDGE: *from what you already own*

Notice that all three buckets are *process* failures, not model failures — which means you can engineer them away. A **written spec** for each agent answers bucket 1. An **explicit hand-off contract** (a structured packet, not a vague summary) answers bucket 2. An **independent reviewer** — ideally an adversary — answers bucket 3. Part D turns these three into a checklist you apply to any system you build.

4.3 What did NOT survive verification

ORIENT: *the big picture before the detail*

A research report is only as honest as the claims it is willing to throw away. Three seductive claims showed up in the search and were killed in the 3-vote refutation pass. Do not repeat them.

Refuted, do not rely on: (1) that hierarchical architectures incur *exponential* token cost — unsupported; (2) that small specialised agents systematically beat one large model — task-dependent, not a rule; and (3) most importantly, that adversarial 'Verifier' agents enable near-zero-error reasoning over million-step chains. That last one is tempting for a validation desk, but the evidence did not hold **V**. **The honest version: adversarial review reduces error; it does not eliminate it, and the magnitude is unproven.** Build your data-integrity and validation agents because they catch real mistakes — not because they promise a guarantee no one has demonstrated.

COMMON MISTAKE: *you might think X; actually Y*

You might think a second agent 'checking' the first is obviously safer. Often yes — but two agents that share the same blind spot will confidently agree on a wrong answer. Verification only helps when the verifier is genuinely independent: different prompt, different evidence, ideally a different model. Redundancy is not independence.

RECAP: *back to altitude*

- Pro: +90.2% on breadth-first research — but vendor eval, breadth-first only, ~15x tokens.
- Con: 41–86.7% failure across 7 production frameworks; gains often minimal vs. baselines.
- Failures cluster in spec (44%), alignment (32%), verification (24%) — all are process, not model, failures.
- Refuted: exponential hierarchy cost, small-beats-large, near-zero-error adversarial chains.
- Verification needs independence, not just a second voice.

SECTION GLOSSARY

MAST: Multi-Agent System Failure Taxonomy; 14 modes in 3 buckets (Cemri et al., NeurIPS 2025).

Breadth-first task: work that splits into many independent strands explorable in parallel. **Best-of-N:** a simple baseline: sample one agent N times and keep the best answer.

Chapter 5. Swarm, deflated — and the framework landscape

BOTTOM LINE ‘Swarm’ in LLM-land means **orchestrated hand-offs, not emergent magic**; OpenAI’s Swarm is explicitly educational and superseded by its Agents SDK; the frameworks differ mostly in how much control they hand you.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Say precisely what ‘swarm’ does and does not mean here.
- Place the major frameworks on a control-vs-convenience spectrum.
- Pick a framework lens for a given job (including ‘use none’).

Prerequisites: Chapter 3.

5.1 The swarm reality-check

ORIENT: *the big picture before the detail*

‘Swarm intelligence’ borrows a glamorous image from ant colonies and bird flocks: simple agents producing emergent global cleverness. In LLM engineering the word has been quietly redefined, and the gap matters.

OpenAI’s **Swarm** framework is, by its own README, an *educational* exploration of ‘ergonomic, lightweight multi-agent orchestration’ built on two primitives: **agents** (instructions + tools) and **handoffs** (an agent can pass the conversation to another at any point) **V**. OpenAI now tells you to migrate to the production **Agents SDK** for real use **V**. So ‘swarm’ here is not emergent self-organisation — it is *orchestrated hand-offs*, the mesh topology with explicit transfer rules. Useful, unmythical, and fully under your control. Treat the biological metaphor as marketing, not mechanism.

COMMON MISTAKE: *you might think X; actually Y*

You might think a ‘swarm’ will self-organise into the right behaviour if you just launch enough agents. It will not. Emergent coordination among LLM agents is a research topic, not a production technique; shipping systems use *explicit* orchestration. More agents without more structure mostly buys you more of the MAST failure modes from Chapter 4.

5.2 The frameworks, on one spectrum

ORIENT: *the big picture before the detail*

You do not need to learn five frameworks. You need to know the axis they vary on: how much low-level control they give you versus how much they do for you.

The landscape, arranged by control vs. convenience:

Framework	Architectural stance
LangGraph	graph of nodes + explicit state; most control, most assembly. Formalises the four patterns of Chapter 3.
AutoGen / AG2	conversational agents that message each other; star/mesh by default.
CrewAI	role-based ‘crew’ abstraction; fast to start, more opinionated.
OpenAI Agents SDK	production successor to Swarm; agents + handoffs, managed.
Claude Agent SDK / Claude Code	subagents + skills + hooks; context-isolation first. What your desk already runs on.

The honest caveat from the research: a rigorous *head-to-head* architectural comparison of these frameworks was not established by the sources **V** — only LangChain’s four-pattern model and Swarm’s primitives were independently confirmed. So choose on fit and on what you can operate, not on a benchmark leaderboard that does not really exist. For your work the choice is already made and correct: you are on the Claude stack, which is context-isolation-first — the property Chapter 4 showed actually matters.

RECAP: *back to altitude*

- ‘Swarm’ = orchestrated hand-offs (mesh + transfer rules), not emergent intelligence.
- OpenAI Swarm is educational; the Agents SDK is its production replacement.
- Frameworks vary on control vs. convenience: LangGraph (most control) → managed SDKs.
- No real head-to-head benchmark exists; choose on fit and operability.
- Your Claude stack is context-isolation-first — the property that matters most.

SECTION GLOSSARY

Swarm (LLM sense): multi-agent coordination via explicit hand-offs; not emergent behaviour.

LangGraph: a graph/state framework giving low-level control over agent flow. **Agents SDK:** OpenAI’s production multi-agent framework, successor to Swarm.

Part C. Building It: The Primitives in Practice

Theory meets the toolchain. This Part is the practitioner’s reference for the orchestration primitives — sub-agents, skills, teams, workflows, loops, persistent memory — and for the two levers that decide whether a system is affordable: **model routing** and **context engineering**. The concepts are tool-agnostic; the running examples come from Claude Code, with the OpenAI Agents SDK and LangGraph noted where they name the same idea differently.

Chapter 6. The orchestration primitives

BOTTOM LINE Five primitives recur across every serious framework — **sub-agents, skills, agent teams, workflows, and loops** — plus **persistent memory**; knowing which one fits which job is most of the craft.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Distinguish sub-agents, skills, teams, and workflows by what they isolate.
- Read and write a sub-agent’s definition (frontmatter as a contract).
- Choose the right primitive for a given job from a decision table.
- Recognise the same primitives across Claude Code, OpenAI, and LangGraph.

Prerequisites: Parts A–B.

6.1 Sub-agents and skills: the two you’ll use most

ORIENT: *the big picture before the detail*

A sub-agent isolates *context*; a skill isolates *instructions*. That one distinction explains when to reach for each.

A **sub-agent** is a worker with its own context window, its own system prompt, and a restricted toolset; it does a focused job and returns a condensed summary to the caller — the lead never sees the worker’s scratch-work. In Claude Code a sub-agent is defined by a markdown file whose frontmatter is the contract; the OpenAI Agents SDK calls the same idea an *agent* with *handoffs*, and LangGraph models it as a *node* with its own state. A **skill** is reusable instructions that load *on demand* — progressive disclosure — so they cost nothing until invoked. Almost every multi-agent system you build will be mostly these two primitives.

a sub-agent definition (Claude Code example)

```
---
name: data-reviewer
description: Adversary vs. data bugs
model: sonnet      # not a judgment agent
tools: Read, Grep, Glob, Bash # read-only
memory: project    # persists across sessions
effort: high
---
You are an adversary. Hunt errors,
leakage, and bad assumptions.
Report; do not fix.
```

COMMON MISTAKE: *you might think X; actually Y*

You might think a sub-agent can talk to its siblings. In most frameworks it cannot — sub-agents report back *only* to the caller. If you need agents to message each other, that is an **agent team** (§6.2), a different primitive.

6.2 Teams, workflows, loops: the three most teams overlook

ORIENT: *the big picture before the detail*

Three primitives are routinely underused. Each closes a specific gap, and Part D will help you decide whether you actually need them.

Agent Teams (experimental): independent full-context sessions with a shared task queue and the ability to *message each other* — for genuinely collaborative work like an adversarial debate between a proposer and a skeptic. **Workflows**: deterministic code that orchestrates dozens of sub-agents (Claude Code caps a workflow at 16 at once, 1,000 total) with loops and branching the model cannot drift

from — the right tool for a batch sweep across hundreds of files. **Loops / routines:** a prompt re-run on an interval (locally) or in the cloud on a cron (1-hour minimum) — for monitoring a long-running job or a daily scan. The two deep-research passes that ground this very book each ran as a workflow: ~110 agents, fan-out search, a 3-vote verification stage, all deterministic.

The decision table — which primitive isolates what:

Primitive	Isolates	Reach for it when...
Skill	instructions	reusable procedure/knowledge, load on demand
Subagent	one context window	a focused task whose detail you don't want in the lead
Agent Team	N full sessions + a mailbox	agents must collaborate / debate
Workflow	state, in a script	large, repeatable, deterministic fan-out
Loop / routine	time	the same job must recur on a schedule

One more primitive deserves a flag: **persistent agent memory**. A single line of configuration (in Claude Code, `memory: project`) gives an agent a directory it reads at the start of every session and maintains across them. Most

people carry cross-session knowledge by hand, in notes *they* tend; persistent memory lets the agent do some of that itself. It is the most overlooked primitive, and Part D returns to when it is worth switching on.

RECAP: back to altitude

- Sub-agent isolates context; skill isolates instructions — the two you'll use most.
- Agent teams add inter-agent messaging; workflows add deterministic large-scale fan-out.
- Loops/routines add scheduled recurrence (local or cloud cron).
- Persistent agent memory (one line of config) is the most overlooked primitive.

SECTION GLOSSARY

Subagent: an isolated-context worker that returns a summary to its caller. **Agent Team:** independent sessions that share a task queue and can message each other. **Workflow:** deterministic script orchestrating many subagents with loops/branches. **Routine:** a cloud-scheduled recurring agent run (1-hour minimum interval).

Chapter 7. Model routing, cost, and context engineering

BOTTOM LINE Two levers decide whether a multi-agent system is affordable: **route each job to the cheapest model that can do it** (Haiku/Sonnet/Opus/Fable), and **treat context as a finite resource** — isolate and compress before it rots.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Right-size a model to a task across the four tiers.
- Explain effort levels and prompt caching as cost levers.
- State what ‘context rot’ is and the two defences against it.
- Decide when a multi-provider router (OpenRouter) earns its place.

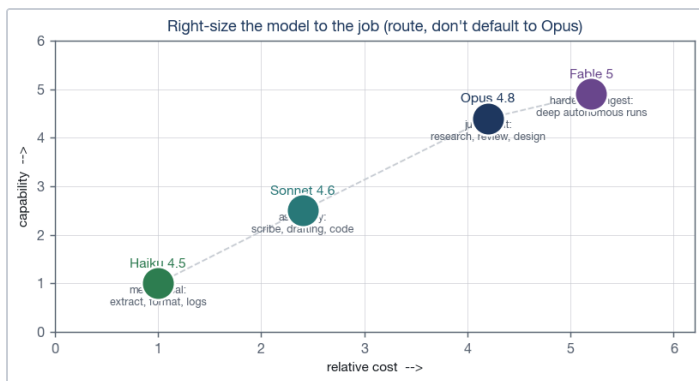
Prerequisites: Chapter 6.

The tiers sort cleanly by job, and every provider has them. **Small / cheap** (e.g. Claude Haiku, GPT-mini) — mechanical work: extract, reformat, parse, collect. **Mid** (Claude Sonnet) — assembly: drafting, routine code, summarising. **Large** (Claude Opus) — judgment: research, review, architecture, anything where being wrong is expensive. **Frontier** (Claude Fable) — the hardest, longest autonomous runs. The single most common waste is leaving every agent on the largest model by default; writing the tier into each agent’s definition is free savings, and most agents do not need the top tier.

7.1 Routing: stop defaulting to the big model

ORIENT: the big picture before the detail

The fastest way to halve an agent system’s cost without losing quality is to stop running every step on the most capable model. Match the tier to the cognitive load.



SCHEMATIC Right-size the model to the job. The dashed line is the cost/capability frontier; pick the lowest tier that clears the task.

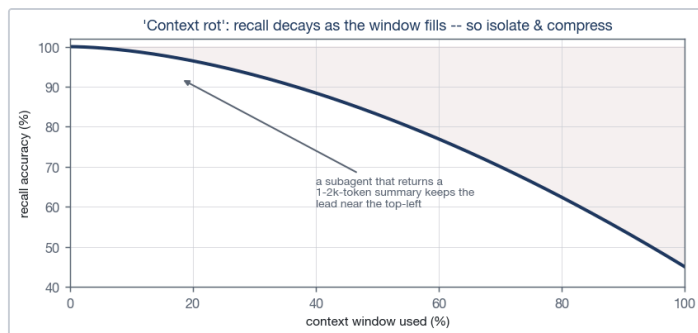
Two more cost levers. **Effort levels** (low→max) dial how much the model deliberates **B**; reserve high/max for the genuinely hard judgment steps and keep mechanical steps low. **Prompt caching** reuses an unchanged context prefix at roughly a 90% discount **B** — but it *invalidates* when you change the model, change effort, toggle fast mode, or re-read a large file mid-session. Practical consequence: keep a stable system prompt and stable early context, and do not thrash the model setting inside a session.

7.2 Context engineering: the finite resource

ORIENT: the big picture before the detail

The deepest idea in production agent design is that the context window is not free space to fill — it is a scarce resource with diminishing returns. Engineers who internalise this build systems that stay sharp; those who do not build agents that get dumber as they work.

Anthropic states it plainly: as the tokens in the window increase, the model's ability to accurately recall any one of them *decreases* **V**. Independent work calls this **context rot** (Chroma, 2025) and **'lost in the middle'** (Liu et al., 2024 — a U-shaped recall curve, with the middle of a long context most likely to be dropped). The two defences are the ones you have already met. **Isolate:** push a sub-task into a subagent with a clean window so the lead never holds the raw detail. **Compress:** have that subagent return a 1–2k-token summary, not its transcript. This is *why* the orchestrator-worker pattern wins when it wins — not because more agents are smarter, but because each one stays in the high-recall zone of its own window.



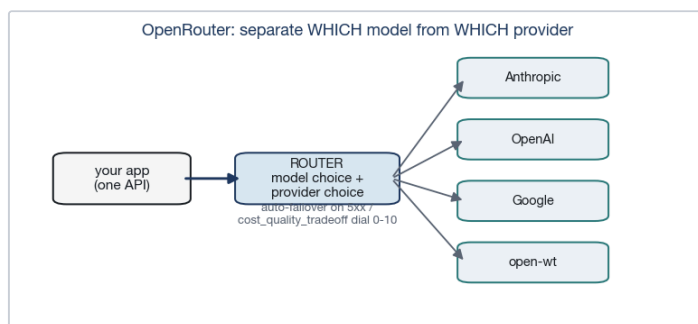
SCHEMATIC Recall decays as the window fills. Isolate work into subagents and return summaries to keep the lead near the top-left.

7.3 Multi-provider routing: when OpenRouter earns its place

ORIENT: the big picture before the detail

Eventually you will want models from more than one provider behind one interface — for price, for resilience, or for a capability only one vendor has. That is what a routing layer like OpenRouter does, and it is worth understanding even before you adopt it.

OpenRouter separates two decisions that are usually tangled: **which model** answers and **which provider** serves it **V**. From that separation you get three things you cannot get from a single vendor: automatic **failover** — on a 5xx or rate-limit it falls through to another provider of the same model **V**; a cost-vs-capability **dial** (`cost_quality_tradeoff`, 0 = most capable ... 10 = cheapest, default 7) plus `:nitro` (fastest) and `:floor` (cheapest) shortcuts **V**; and an **Auto Router** that picks a model per prompt by estimated complexity. For your desk this is a *later* lever, not an urgent one — you are productive on the Claude stack and the no-spend rule governs paid routing — but it is the right tool the day you want provider resilience or a cheaper model for the mechanical 80% of calls.



SCHEMATIC A multi-provider router separates model choice from provider choice, adding failover and a cost/quality dial.

COMMON MISTAKE: *you might think X; actually Y*

You might think routing across providers is mainly about chasing the cheapest token. The bigger win is usually *resilience* — one provider's outage stops being your outage — and *access* to a capability a single vendor lacks. Price is the third reason, not the first.

RECAP: *back to altitude*

- Route by cognitive load: Haiku (mechanical), Sonnet (assembly), Opus (judgment), Fable (hardest).
- Effort and prompt caching are cost levers; caching invalidates on model/effort/fast-mode changes.
- Context rot is real: recall falls as the window fills; isolate + compress are the defences.
- Orchestrator-worker wins by keeping each agent in its high-recall zone — not by adding brains.
- Multi-provider routing (OpenRouter) buys failover and access first, price second — a later lever.

SECTION GLOSSARY

Effort level: a dial (low...max) on how much a model deliberates before answering. **Prompt caching:** reuse of an unchanged context prefix at ~90% discount; invalidated by config changes. **Context rot:** the decline in recall accuracy as a context window fills.

OpenRouter: a layer that routes one API across many models/providers with failover and cost dials.

Part D. Designing Your Own Stack

Theory is only useful if you can apply it. This Part turns the whole book into a repeatable **method**: how to look at your own work, decide where an agent system actually helps, find the real gaps in what you have, sequence the build, and wrap it in the governance that prevents failure. It is the part to return to whenever you start a new project — a checklist for architecting deliberately instead of by accident.

Chapter 8. A five-step method for architecting any agent system

BOTTOM LINE Architect deliberately: (1) map your work to a lifecycle, (2) apply the escalation ladder to each stage, (3) find your real gaps by asking *which limit am I actually hitting*, (4) sequence fixes by impact over effort, (5) wrap it all in a governance layer of specs, hand-offs, and verification.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Run the five-step method on a project of your own.
- Decide, per stage, whether it needs one agent, a tool, or many agents.
- Diagnose a gap by the limit it relieves, not the feature it adds.
- Add the three governance artefacts that prevent most failures.

Prerequisites: Parts A–C.

8.1 Step 1 — map your work to a lifecycle

ORIENT: *the big picture before the detail*

Almost any knowledge work decomposes into a short lifecycle: explore the space, gather the material, build something, verify it, ship it. Naming your stages is the first design act, because the right architecture is different for each one.

Write down the stages your work actually has. A researcher's lifecycle might be explore → gather → analyse → verify → report; a writer's, ideate → research → draft → edit → publish; a developer's, scope → build → test → review → release. The exact shape barely matters; what matters is that you now have named stages you can architect *separately*, because the next steps treat each one on its own terms.

Step 1: map your work to a lifecycle



Any knowledge work decomposes into a short lifecycle -- architect each stage on its own terms.
spend multi-agent complexity at the breadth-first ends (explore, gather); keep the middle lean

SCHEMATIC Any knowledge work decomposes into a short lifecycle.
Architect each stage on its own terms.

8.2 Step 2 — apply the escalation ladder to each stage

ORIENT: *the big picture before the detail*

For every stage you just named, ask the cheapest question first. Most stages do not need a multi-agent system; some do not even need an agent.

Walk each stage up the ladder from Chapter 2. Would a **single agent with a good prompt** do? If it struggles, does a **better tool** fix it before a second agent would? Only if a stage hits a real limit — context pressure, genuine parallelism, or distinct expertise — does it earn **sub-agents** or full **orchestration**. The breadth-first stages (explore, gather) usually justify the climb; the sequential ones (build, report) usually do not. Spend complexity where the work is parallel and high-value; keep the rest lean.

8.3 Step 3 — find gaps by the limit they relieve

ORIENT: *the big picture before the detail*

When you audit an existing setup, the temptation is to list missing features. Resist it. A feature is only a gap if it relieves a limit you are actually hitting.

For each capability you are *not* using — persistent memory, scheduled runs, deterministic workflows, multi-agent teams, multi-provider routing — ask the diagnostic question: **which limit would this relieve, and am I hitting it?** Persistent memory earns its place only if you are re-teaching the system the same thing every session. Workflows earn theirs only if you run the same large fan-out repeatedly. A capability with no matching limit is not a gap; it is a distraction. Sort the real gaps into two piles — *leverage* (capability you have not switched on) and *hygiene* (small decay to clean up) — because the next chapter sequences them differently.

COMMON MISTAKE: *you might think X; actually Y*

You might think a good audit produces a long list of everything you could add. A good audit produces a *short* list of what relieves a limit you actually feel. Length is a warning sign: it usually means you are cataloguing the platform, not diagnosing your work.

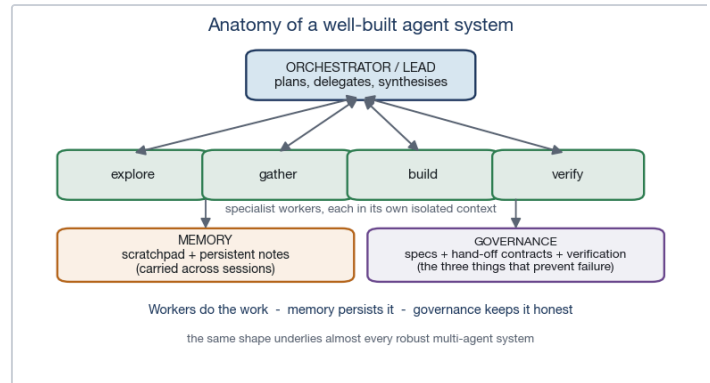
8.4 Steps 4 & 5 — sequence, then add governance

ORIENT: *the big picture before the detail*

Two steps remain, and the second is the one most people skip — which is exactly why their multi-agent systems fail in the ways Chapter 4 documented.

Step 4: sequence by impact over effort (the next chapter gives you the grid). **Step 5: wrap the whole thing in a governance layer.** This is the non-negotiable one. Recall the three buckets that account for most multi-agent failures: vague specs, dropped hand-offs, and unverified

output. The cure for each is a deliberate artefact — **specs** (a written, testable description of what each agent must do), **hand-off contracts** (an explicit structured packet passed between stages, not a vague summary), and **verification** (an independent check, ideally an adversary, before output is trusted). A system with these three is robust almost regardless of its topology; a system without them is fragile regardless of how clever its agents are.



SCHEMATIC The target shape: an orchestrator + isolated specialist workers + memory + a governance layer of specs, hand-offs, and verification.

RECAP: *back to altitude*

- Five steps: map the lifecycle, apply the ladder, find gaps by limit, sequence, add governance.
- Most stages need one agent or a tool — not a multi-agent system.
- A feature is a gap only if it relieves a limit you are actually hitting.
- Governance = specs + hand-off contracts + verification; it answers the three failure buckets.

SECTION GLOSSARY

Lifecycle: the named stages your work decomposes into; architect each separately. **Real gap:** a capability that relieves a limit you are actually hitting — not a missing feature. **Governance layer:** the specs, hand-off contracts, and verification that make a system robust.

Chapter 9. Sequencing the build, and the principle that scales

BOTTOM LINE Sequence upgrades by **impact over effort** — free wins first, deliberate bets later — map each lifecycle stage to the cheapest primitive that fits, and hold above all to one principle: *restraint with reasons*.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Use an impact/effort grid to order your build.
- Match each lifecycle stage to a primitive and a model tier.

- State the principle that separates senior judgment from gadget-chasing.

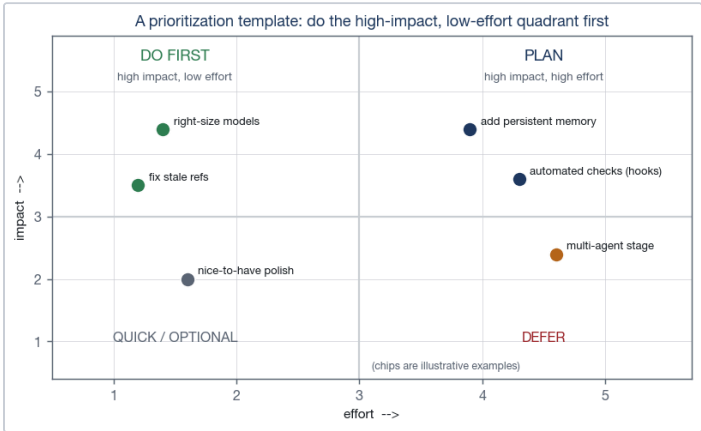
Prerequisites: Chapter 8.

9.1 Sequence by impact over effort

ORIENT: the big picture before the detail

Once you have a short list of real gaps, you have a finite amount of attention to spend on them. Order them the way an investor sizes positions: by payoff against the effort to capture it.

Drop each candidate change onto a simple grid: impact up, effort right. The **high-impact, low-effort** quadrant is your do-first list — usually configuration and hygiene (right-sizing models, fixing stale references) that pay off immediately. **High-impact, high-effort** changes (a new memory system, automated checks, a multi-agent stage) are planned, not impulsive. Low-impact work waits regardless of effort. The grid is not science; it is a way to refuse to do the shiny thing before the useful one.



SCHEMATIC Sequence your build by impact over effort. Do the high-impact, low-effort quadrant first.

9.2 Match the primitive to the stage

ORIENT: the big picture before the detail

With the lifecycle from Chapter 8 in hand, assign each stage the cheapest primitive that fits — the heart of an affordable, robust design.

The mapping has the same shape for most knowledge work. **Exploration** is breadth-first — the prime case for parallel sub-agents, and the one place the 90.2% result most applies. **Gathering material** often fans out too. **Verification** wants an independent adversary, not a yes-man — with the honest expectation from Chapter 4 that it *reduces* error rather than eliminating it. **Building** and **reporting** are usually single coherent efforts — one capable agent, not a committee that adds seams. Match the primitive to the stage and you spend tokens where they earn and save them where they would not.

A generic mapping you can adapt:

Stage	Primitive	Model tier
Explore / ideate	parallel sub-agents	large lead / mid workers
Gather material	fan-out sub-agents or a workflow	small / cheap workers
Build / draft	single agent	mid
Verify / review	independent adversary (or a team)	large
Report / ship	single agent	mid

9.3 The principle that scales

ORIENT: the big picture before the detail

Tools change every quarter; one principle does not. If you remember a single thing from this book, make it this one.

The principle is **restraint with reasons**: use the simplest thing that works, and escalate only when the evidence justifies the cost. A single agent where a single agent suffices; multi-agent only where the work is parallel and the value is high; a verification step on anything you will act on. This is what separates durable systems from impressive demos — and, not coincidentally, what separates senior engineering judgment from gadget-chasing. Anyone can wire up more agents. The skill is knowing *which* pattern, *why*, at *what cost*, and *where it fails*.

COMMON MISTAKE: you might think X; actually Y

You might think the most impressive system is the most automated one. The most impressive system is the one that is exactly as complex as its problem and no more. Over-automation reads as inexperience; calibrated restraint reads as mastery.

RECAP: back to altitude

- Sequence by impact over effort: free wins first, deliberate bets later.
- Map each stage to the cheapest primitive that fits; verification gets an independent check.
- The principle that scales: restraint with reasons.
- Skill is knowing which pattern, why, at what cost, and where it fails — not adding agents.

SECTION GLOSSARY

Impact/effort grid: a 2×2 for ordering changes: do high-impact, low-effort first. **Restraint with reasons:** use the simplest thing that works; escalate only on evidence. **Independent verification:** a check by a different agent / model / prompt, not a second identical pass.

Part E. How the Pros Work

Theory and tooling are necessary; they are not the same as knowing what actually works in practice. This Part distils what leading teams really do — led by the one habit that separates durable systems from demos: **verification**. To make the case vivid it uses a hard-won case study from finance, where the failure mode is unusually easy to measure — then generalises the lesson to any domain, and closes with the meta-practices that scale. It is also honest about where the evidence ran out.

Chapter 10. A case study from finance: don't let the model be the answer

BOTTOM LINE In a clean test, LLM trading agents show **no real edge (all $p > 0.34$)** and ~37%/19% of their apparent skill is **memorisation** — a vivid, measurable example of why you must *verify* model output and why a bigger model makes it worse, not better. The lesson generalises to any domain.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- State the FINSABER null result and why prior 'LLM edge' was fragile.
- Explain why memorisation IS look-ahead bias in an LLM backtest.
- Steal the four validated defences, and their general form.
- Carry the lesson — verify, and don't trust a bigger model — into your own domain.

Prerequisites: Parts A–D.

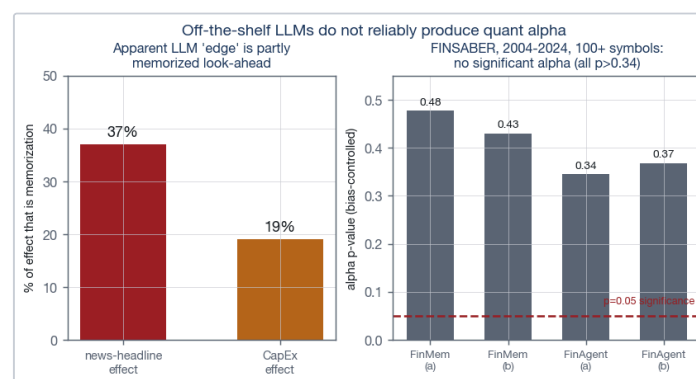
10.1 The null result, stated plainly

ORIENT: *the big picture before the detail*

The most important thing the pros' literature has to say to a quant is a warning, not a recipe. The instinct 'point a clever LLM at the market and harvest alpha' has been tested carefully, and it fails.

The **FINSABER** study back-tested LLM investing agents (FinMem, FinAgent) over 2004–2024 across 100+ symbols with explicit controls for survivorship, look-ahead, and data-snooping. The result: **neither agent generated statistically significant alpha — every p-value exceeded 0.34** 🟢. The authors are blunt about why earlier papers looked better: they used 'only a small number of well-known stocks such as TSLA and AMZN' — historical winners — over short windows, baking in survivorship and look-

ahead. The edge was an artefact of the setup, and it vanished under a broader, longer, honest test.



🟢 The null result. Left: how much of the apparent effect is memorisation. Right: FINSABER's bias-controlled alpha p-values — none significant.

10.2 Why: memorisation is look-ahead by another name

ORIENT: *the big picture before the detail*

The mechanism behind the null is the single most important idea for anyone putting an LLM near market data. It is a bias you already hunt — wearing a new disguise.

A pre-trained LLM has read the future. Ask it about a 2019 event and it may 'recall' what happened next, because the consequences were in its training corpus. So a backtest over any period inside the training cut-off is *effectively in-sample* 🟢. The magnitude is not small: about **37% of a news-headline return effect and 19% of a CapEx effect** trace to memorisation rather than reasoning 🟢, and the inflated

accuracy collapses out-of-sample. This is **look-ahead leakage** — the classic failure careful analysts guard against with point-in-time data — except here the leak is hidden inside the model’s weights, where an ordinary audit of your inputs cannot see it. The same trap waits in any field where the model may have read the outcome: case law, medical results, sports, elections.

COMMON MISTAKE: *you might think X; actually Y*

You might think a bigger, smarter model fixes this. It makes it *worse*: a larger model has memorised more of the past, so its backtest looks even stronger and is even more contaminated. The fix is never a better model — it is methodology: bias controls, point-in-time data, and out-of-sample discipline.

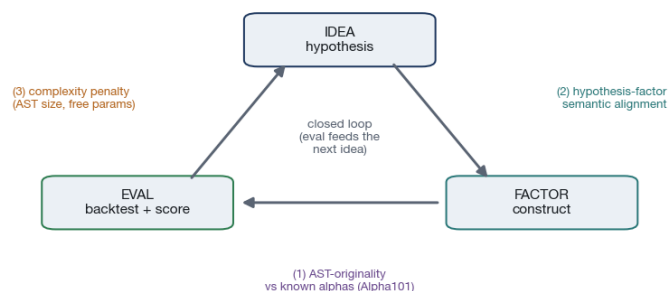
10.3 The four defences worth stealing

ORIENT: *the big picture before the detail*

The same literature that delivers the warning also delivers the countermeasures. Four are concrete and cheap — and each has a general form you can carry into any domain.

(1) LAP — a memorisation diagnostic. Lookahead Propensity, built on the MIN-K% membership-inference statistic, scores how ‘familiar’ a prompt’s rarest tokens are to the model; a positive prediction×LAP interaction flags and *quantifies* look-ahead with no retraining **✓**. **(2) Point-in-time decay testing.** Measure performance decay across temporally distinct regimes against quantitative baselines — genuine prediction holds, memorised ‘edge’ decays **✓**. **(3) AlphaAgent — regularised multi-agent mining.** A closed Idea→Factor→Eval loop with three guards: AST-originality against known alphas (Alpha101), hypothesis–factor semantic alignment, and an AST/complexity penalty — because unconstrained LLMs homogenise toward documented factors like RSI and over-engineer overfit-prone constructions **✓**. **(4) AlphaEval — five-axis evaluation.** Score a candidate on predictive power, stability, robustness, financial logic, and diversity, not correlation alone **✓**.

AlphaAgent (KDD 2025): regularized multi-agent alpha mining



✓ AlphaAgent’s closed loop with three anti-overfitting regularisers — a directly stealable multi-agent blueprint.

BRIDGE: *from what you already own*

Strip the finance vocabulary and the four defences are universal. (1) A **memorisation / originality check** — does the model already ‘know’ the answer? — applies to any task where the outcome might be in training data. (2) **Out-of-sample / holdout discipline** — test on data the model could not have seen — is the single most important habit in *any* evaluation. (3) **Complexity penalties** — prefer the simpler hypothesis — guard against overfitting everywhere. (4) **Multi-axis evaluation** — score on more than one metric — catches fragility a single number hides. Whatever you build, build these four in.

RECAP: *back to altitude*

- FINSABER: in a clean test, LLM trading agents produce no real edge (all $p > 0.34$).
- ~37%/19% of apparent LLM skill is memorisation = hidden look-ahead leakage.
- Bigger models memorise more — the fix is methodology, not model size.
- Four general defences: originality check, holdout discipline, complexity penalty, multi-axis eval.
- The trap is any domain where the model may have read the outcome.

SECTION GLOSSARY

FINSABER: a bias-controlled benchmark showing LLM trading agents yield no significant alpha. **LAP / MIN-K%:** a cheap membership-inference diagnostic that flags LLM memorisation / look-ahead. **AlphaAgent:** a regularised Idea→Factor→Eval multi-agent alpha-mining loop (KDD 2025). **Point-in-time:** data/evaluation that exposes the model only to information available at the time.

Chapter 11. Meta-practices the best teams share — and an honest gap

BOTTOM LINE Four practitioner habits recur in the strongest sources — **scale agents to complexity, engineer context as a scarce resource, keep a human in the verification loop, and start evals small (20–50 real failures)**; the writing and business workflows did not survive verification, which is itself the lesson.

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Apply the 1 / 2–4 / >10 agent-count heuristic.
- Engineer context with just-in-time retrieval and persistent NOTES.md memory.
- Bootstrap an eval suite from 20–50 real failures.
- State honestly which domains the evidence did NOT reach.

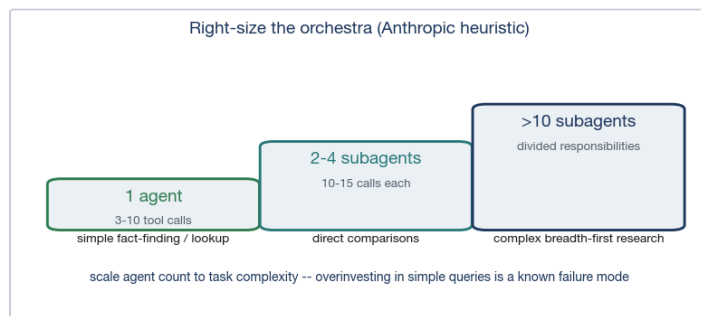
Prerequisites: Chapter 6; Chapter 7.

11.1 Right-size the orchestra

ORIENT: *the big picture before the detail*

The best agent teams are stingy with agents. They have explicit rules for how many to spawn, because over-spending on simple queries was an early, expensive failure mode.

Anthropic’s published heuristic is worth memorising: **1 agent** with 3–10 tool calls for simple fact-finding; **2–4 subagents** for direct comparisons; **>10 subagents** only for genuinely complex, breadth-first research (V). This is Chapter 2’s pragmatic-escalation rule made quantitative — and it is the discipline that keeps the 15× multi-agent token bill (Chapter 4) from landing on tasks that never needed it.



(V) Scale agent count to task complexity. Overinvesting in simple queries is a named failure mode.

11.2 Context as a scarce resource

ORIENT: *the big picture before the detail*

Chapter 7 taught context rot as theory. The practitioners turn it into two concrete habits you can adopt this week.

Just-in-time retrieval. Do not pre-load everything into the window. Keep lightweight identifiers — file paths, stored

queries, links — and pull data in at runtime with tools (head/tail/grep/glob) (V). **Persistent external memory.** For long-horizon work, have the agent write structured notes to a NOTES.md outside the context window and re-read them across resets (V). This is the practitioner form of the persistent-memory primitive from Chapter 6: the best teams externalise memory deliberately rather than carrying it by hand. (One honest nuance: Anthropic’s actual recommendation is a *hybrid* — some pre-loading for speed plus just-in-time exploration — not pure retrieval (V).)

11.3 Start evals small, keep a human in the loop

ORIENT: *the big picture before the detail*

The habit that separates durable systems from demos is verification — and the barrier to it is far lower than teams assume.

Teams routinely delay building evaluations because they think they need hundreds of cases. Anthropic’s guidance: **20–50 simple tasks drawn from real failures** — your bug tracker, support queue, or a log of times the system got it wrong — is a great start (V). You almost certainly already have the raw material; the cases where it failed are exactly what an eval suite is built from. The finance case study makes this non-negotiable: because the dangerous output is *confidently wrong* and looks great until it ships, a human-in-the-loop verification step is not optional polish — it is the thing standing between you and a quiet failure in production.

11.4 The honest gap

ORIENT: *the big picture before the detail*

A research pass is only trustworthy if it reports what it failed to find. This one has a real hole, and naming it is part of the discipline this whole book argues for.

The two research tracks behind this Part returned strong, verified material on quant practice and on cross-cutting meta-practices. They returned **nothing that survived adversarial verification on the writing and business domains** (V) — no editorial-pipeline pattern, voice-preservation method, or lean-startup-with-AI workflow cleared the 3-vote bar. The correct reading is *‘not established here,’* not *‘no good practice exists.’* The likely truth is that

writing and business work should inherit the *same* meta-practices — small evals from real failures, human-in-the-loop review, context discipline, model routing — rather than any domain-specific trick; but this research did not prove that, and this book will not pretend it did. That refusal to over-claim is the habit, not an aside.

COMMON MISTAKE: *you might think X; actually Y*

You might think an empty result is a failed search. It is a finding. ‘We looked hard and the verified evidence is thin here’ tells you exactly where to be cautious and where to run your own test — which is far more useful than a confident summary built on sources that would not survive scrutiny.

RECAP: *back to altitude*

- Agent-count heuristic: 1 (simple) / 2–4 (comparisons) / >10 (complex breadth-first).
- Context habits: just-in-time retrieval + persistent NOTES.md (hybrid, not pure retrieval).
- Evals start at 20–50 real failures — you already have them in kill-log/corrections.
- Writing & business workflows did NOT survive verification: ‘not established,’ not ‘absent.’
- Reporting an empty result honestly IS the meta-practice.

SECTION GLOSSARY

Just-in-time retrieval: loading data at runtime via identifiers rather than pre-filling context. **NOTES.md memory:** structured notes persisted outside the window for long-horizon coherence. **Eval suite:** a small set of real-failure tasks used to verify agent behaviour; start at 20–50. **Human-in-the-loop:** a required human verification checkpoint before output is trusted/acted on.

Part •. Where This Leaves Us

The verdict in one sentence: modern AI architecture is a three-layer stack whose multi-agent top is powerful but task-conditional — start single-agent, add tools before agents, escalate only on evidence, isolate context, route models by cost, verify everything you act on, and wrap the whole thing in specs, hand-offs, and an independent check.

The evidence base. This guide is built on adversarially fact-checked research: two deep-research passes on the agent-AI literature **V** (53 sources total, several claims — including flattering ones — refuted and dropped), plus a docs-level reference of the major agent frameworks **B**. Headline numbers — +90.2% / ~15× / 41–86.7% / MAST 44.2-32.3-23.5 / FINSABER $p > 0.34$ / 37%-19% memorisation — are verified; diagrams and worked cases are schematic. **The deepest lesson is the one the whole book returns to:** an agent system is only as trustworthy as its verification, and the most senior thing you can do is use the simplest architecture that solves your problem — no more. Build deliberately, verify honestly, and escalate only on evidence. That is the whole craft.